

FINAL PROJECT

Student: Trey Pickens

Student ID: ber221

Course:

MAT 2253 Applied Linear Algebra

Section: 003

Due Date:

Wed, Dec 10th, 2025

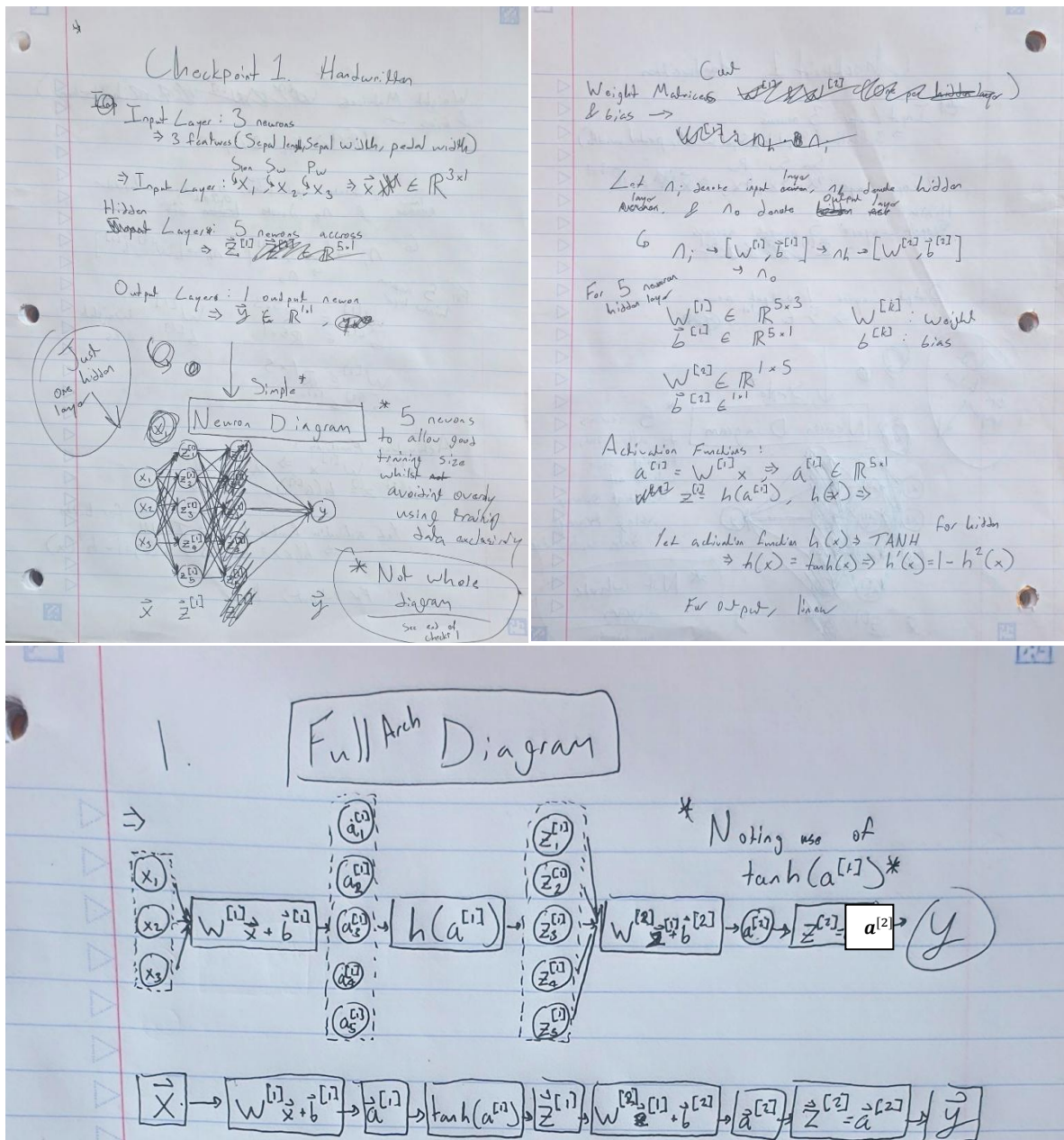
Table of Contents

Table of Contents.....	2
Checkpoint 1.....	3
1.1. Architecture Diagram.....	3
1.2. Dimension Verification Table:	4
1.3. Weight Initialization.....	4
1.4. Justification for Hidden Layer Size.....	5
Checkpoint 2.....	6
2.1. Data Point Verification	6
2.2. Hidden Layer Pre-Activations	6
2.3. Hidden Layer Activations	8
2.4. Output Layer Computation.....	9
2.5. Loss Computation.....	10
2.6. Verification Table.....	12
Checkpoint 3.....	13
3.1. Output Layer Gradient Derivation	13
3.2. Gradient for $W[2]$	14
3.3. Gradient for $b[2]$	15
3.4 Error Signal at Hidden Layer.....	15
3.5. Activation Function Derivative.....	17
3.6. Gradient With Respect to Pre-Activations	18
3.7. Gradient for $W[1]$	19
3.8. Gradient for $b[1]$	20
3.9. Complete Chain Rule Example.....	20
3.10. Verification Table.....	22
Notebook Summary Screenshot(s)	24
Appendices	25
Appendix A. Handwritten Arithmetic for Forward Pass.....	25
Appendix B. Handwritten Derivative Work for Backpropagation.....	26
Appendix C. Complete Notebook Run	28

Checkpoint 1.

1.1. Architecture Diagram

The collection of images below elaborates on the pieces of our neural network. Our neural network consists of an *input* layer, vector $x \in \mathbb{R}^{3 \times 1}$, with 3 features, those being the *sepal length*, *sepal width*, and *pedal width* of a plant (with positions in x being x_1 , x_2 , and x_3 respectively), as well as showing our *hidden* layer $z \in \mathbb{R}^{5 \times 1}$ and our output layer $y \in \mathbb{R}^{1 \times 1}$, which is essentially just our single predicted *pedal length*. The final photo is the full diagram.



In the diagram, it also shows our *weight matrices*, *bias vectors*, and *activation vectors*, those being $W^{[1]}$, $W^{[2]}$, $b^{[1]}$, $b^{[2]}$, $a^{[1]}$, & $a^{[2]}$, though $a^{[2]} = z^{[2]} = y$, since the transition into the output layer is linear. We also see our chosen *activation function* $h(z) = \tanh z$.

1.2. Dimension Verification Table:

Operation	Left Operand	Right Operand	Result
$a^{[1]} = W^{[1]}x$	$W^{[1]} \in \mathbb{R}^{5 \times 3}$	$x \in \mathbb{R}^{3 \times 1}$	$a^{[1]} \in \mathbb{R}^{5 \times 1}$
$z^{[1]} = h(a^{[1]})$	$a^{[1]} \in \mathbb{R}^{5 \times 1}$	(Element-wise operation)	$z^{[1]} \in \mathbb{R}^{5 \times 1}$
$a^{[2]} = W^{[2]}z^{[1]}$	$W^{[2]} \in \mathbb{R}^{1 \times 5}$	$z^{[1]} \in \mathbb{R}^{5 \times 1}$	$a^{[2]} \in \mathbb{R}^{1 \times 1}$
$y = a^{[2]}$	$a^{[2]} \in \mathbb{R}^{1 \times 1}$	(Linear Relation, single value)	$y \in \mathbb{R}$

1.3. Weight Initialization

Utilizing the provided notebook file to randomly initiate our weights, we gain the following values using our **seed, which is 12** (based on my student ID):

3. Initialized weights

$W^{[1]} \in \mathbb{R}^{5 \times 3}, W^{[1]} = \begin{bmatrix} -0.026 & -0.0556 & 0.5209 \\ -0.6284 & -0.3727 & -0.8555 \\ -0.1029 & -0.1173 & 0.5641 \\ -0.0063 & -0.3066 & 0.6868 \\ 0.8055 & -0.3446 & 0.346 \end{bmatrix}$

$b^{[1]} \in \mathbb{R}^{5 \times 1}, b^{[1]} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, b^{[1]} = \vec{0}^{5 \times 1}$

~~W^{[2]} \in \mathbb{R}^{1 \times 5}, W^{[2]} = \begin{bmatrix} -0.2241 & 0.0812 & 0.1286 & -0.6377 & 0.032 \end{bmatrix}~~

$b^{[2]} \in \mathbb{R}^{1 \times 1}, b^{[2]} = [0]$

Below is the **notebook output** providing the randomized data (Notebook Cell 3):

```
=====
INITIALIZED WEIGHTS AND BIASES
=====

-----
W^(1): Weights from Input to Hidden Layer
-----
NumPy shape:      (5, 3)
Mathematical shape: 5 × 3

Values (rounded to 4 decimals):
[[-0.026 -0.0556  0.5209]
 [-0.6284  0.3727 -0.8555]
 [-0.1029 -0.1173  0.5641]
 [-0.0063 -0.3066  0.6868]
 [ 0.8055 -0.3446  0.346  ]]

-----
b^(1): Bias for Hidden Layer
-----
NumPy shape:      (5,)
Mathematical shape: 5 × 1 (column vector)
Values: [0. 0. 0. 0. 0.]

-----
W^(2): Weights from Hidden to Output Layer
-----
NumPy shape:      (1, 5)
Mathematical shape: 1 × 5

Values (rounded to 4 decimals):
[[-0.2241  0.0812  0.1286 -0.6377  0.032  ]]

-----
b^(2): Bias for Output Layer
-----
NumPy shape:      (1,)
Mathematical shape: 1 × 1 (scalar)
Values: [0.]
```

1.4. Justification for Hidden Layer Size

Importantly, our **hidden layer size**, that being **5 neurons**. Given that the number of input neurons we have is 3, we want to ensure that the number of neurons in the hidden layer is greater. However, we also want to ensure that we avoid overfitting. 6 neurons may be dangerous in terms of overfitting due to its size (double the input), so we reduce it to 5 to ensure that we still have enough for complex regression while reducing the overfitting risk.

Checkpoint 2.

2.1. Data Point Verification

The first datapoint X and observed outcome y we are utilizing is $X = \begin{bmatrix} 5.1 \\ 3.5 \\ 0.2 \end{bmatrix}$, $y = 1.4$

2.2. Hidden Layer Pre-Activations

The first step is to get to the **pre-activation vector** by computing $a^{[1]} = W^{[1]}x + b$. Bias vector b adds nothing as it is a zero vector.

~~Handwritten:~~
First data point: $x = \begin{bmatrix} 5.1 \\ 3.5 \\ 0.2 \end{bmatrix}$ $y = 1.4$

* $W^{[1]}, b^{[1]}$ from Checkpoint 1 *

$a^{[1]} = W^{[1]}x =$

$b^{[1]} = \vec{0}$, not needed

Weight

Data point

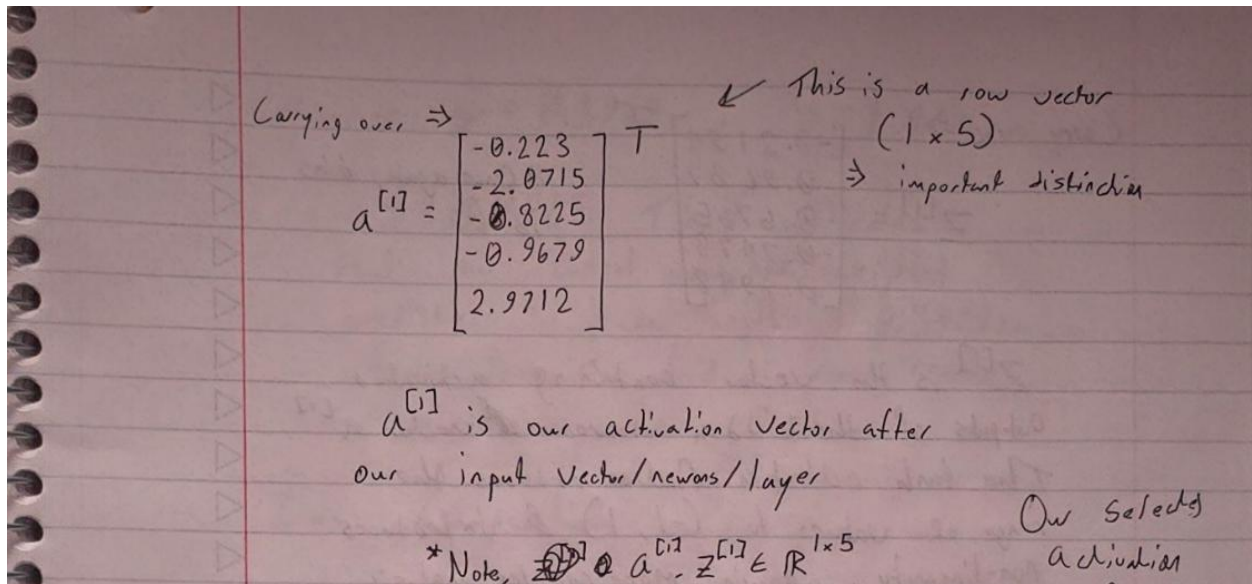
$\begin{bmatrix} -0.026 & -0.0556 & 0.5209 \\ -0.6284 & 0.3727 & -0.8555 \\ -0.1029 & -0.1173 & 0.5641 \\ -0.0063 & -0.3066 & 0.6868 \\ 0.8055 & -0.3446 & 0.346 \end{bmatrix} \begin{bmatrix} 5.1 \\ 3.5 \\ 0.2 \end{bmatrix}$

* Important note: *

Coming back to check, some values only match 2 decimal places, likely due to rounding rules, but our formula was valid *

$= \begin{bmatrix} (-0.026)(5.1) + (-0.0556)(3.5) + (0.5209)(0.2) \\ (-0.6284)(5.1) + (0.3727)(3.5) + (-0.8555)(0.2) \\ (-0.1029)(5.1) + (-0.1173)(3.5) + (0.5641)(0.2) \\ (-0.0063)(5.1) + (-0.3066)(3.5) + (0.6868)(0.2) \\ (0.8055)(5.1) + (-0.3446)(3.5) + (0.346)(0.2) \end{bmatrix}^T$

$= \begin{bmatrix} -0.1326 & -0.1946 & 0.10418 \\ -3.20484 & 1.30445 & -0.1711 \\ -0.52479 & -0.41055 & 0.11282 \\ -0.03213 & -1.0731 & 0.13736 \\ 4.10805 & -1.2061 & 0.0692 \end{bmatrix}^T \approx \begin{bmatrix} -0.223 \\ -2.0715 \\ -0.8225 \\ -0.9679 \\ 2.9712 \end{bmatrix}^T$



Notebook Cell 5 Output:

Dimensional Analysis

$W^{(1)}$: 5 x 3 (mathematical), shape (5, 3) (NumPy)

x : 3 x 1 (mathematical), shape (3,) (NumPy)

$b^{(1)}$: 5 x 1 (mathematical), shape (5,) (NumPy)

$a^{(1)}$: 5 x 1 (mathematical), shape (5,) (NumPy)

Pre-Activation Values $a^{(1)}$

Values (rounded to 4 decimals):

$[-0.2229 \ -2.0714 \ -0.8226 \ -0.9679 \ 2.9711]$

Interpretation

Each $a_i^{(1)}$ is the weighted sum of inputs to hidden neuron i .
These values are BEFORE applying the activation function.

The resulting matrix is our **pre-activation vector** $a^{[1]}$, which is a *row vector*. This is the vector that our *activation function* will use to calculate the *hidden layer vector* $z^{[1]}$.

It is important to mention that, despite the request that the accuracy of the values between *my calculations* and the provided *notebook output* be at least three decimal places, $a_1^{[1]}$ only matches two. It is still accurate and likely a difference of rounding standards between the handwritten arithmetic and the usual rounding conventions found in computers. This is evidenced by the very small difference of **.0001** between them as well as maintained accuracy of later calculations.

2.3. Hidden Layer Activations

Utilizing our **pre-activation vector** $a^{[1]}$, we can apply our previously determined **activation function** $h(z) = \tanh z$ to obtain $z^{[1]}$

Our selected
activation
function

* Note, $a^{[1]}, z^{[1]} \in \mathbb{R}^{1 \times 5}$

Hidden Layer Activation ($a^{[1]} \Rightarrow z^{[1]}$)

for all $z_i^{[1]}, a_i^{[1]}, z_i^{[1]} = \tanh(a_i^{[1]})$

$$= \frac{e^{a_i^{[1]}} - e^{-a_i^{[1]}}}{e^{a_i^{[1]}} + e^{-a_i^{[1]}}}$$

$$\left. \begin{aligned} z_1^{[1]} &= \frac{-0.4497}{2.0499} = -0.2194 \\ z_2^{[1]} &= \frac{-7.8107}{8.0627} = -0.9687 \\ z_3^{[1]} &= \frac{-1.8369}{2.7155} = -0.6765 \\ z_4^{[1]} &= \frac{-2.2525}{3.0123} = -0.7478 \\ z_5^{[1]} &= \frac{19.4640}{19.5666} = 0.9948 \end{aligned} \right\} z^{[1]} = \begin{bmatrix} -0.2194 \\ -0.9687 \\ -0.6765 \\ -0.7478 \\ 0.9948 \end{bmatrix}$$

$\uparrow$
 $h(a^{[1]}) = \tanh(a_i^{[1]})$

Notebook Cell 6:

```
-----  
Dimensional Analysis  
-----  
Input  a^(1): 5 x 1 (mathematical), shape (5,) (NumPy)  
Output z^(1): 5 x 1 (mathematical), shape (5,) (NumPy)  
  
-----  
Activation Values z^(1)  
-----  
Values (rounded to 4 decimals):  
[-0.2193 -0.9687 -0.6765 -0.7478  0.9948]
```

$z^{[1]}$ is the vector containing the activated outputs of the *pre-activation* vector with values reduced to the range (-1, 1). This also introduces non-linearity, allowing for more complex analysis.

2.4. Output Layer Computation

Since we are moving to the output layer, our *activation function* is linear, meaning it will not need to be computed separately, and the values may be treated as the same (while maintaining an understanding of where they come from).

Output Layer Computation

Since we are moving to the output layer, the activation layer is linear, so $a^{[2]} = z^{[2]} = \hat{y}$

Thus, $W^{[2]} z^{[1]} = \hat{y}$

$$= \begin{bmatrix} -0.2241 & 0.0812 & 0.1286 & -0.6377 & 0.032 \end{bmatrix} \begin{bmatrix} -0.2194 \\ -0.9687 \\ -0.6765 \\ -0.7478 \\ 0.9948 \end{bmatrix}$$
$$= \begin{bmatrix} (-0.2241)(-0.2194) + (0.0812)(-0.9687) + (0.1286)(-0.6765) + (-0.6377)(-0.7478) + (0.032)(0.9948) \end{bmatrix}$$
$$= \begin{bmatrix} 0.0492 - 0.0787 - 0.087 + 0.4769 + 0.0318 \end{bmatrix}$$
$$= \begin{bmatrix} 0.3922 \end{bmatrix}$$
$$\Rightarrow a^{[2]} = \begin{bmatrix} 0.3922 \end{bmatrix} \Rightarrow \hat{y} = 0.3922$$

Notebook Cell 7 Output:

----- Dimensional Analysis

W^(2): 1 × 5 (mathematical), shape (1, 5) (NumPy)

z^(1): 5 × 1 (mathematical), shape (5,) (NumPy)

b^(2): 1 × 1 (scalar), shape (1,) (NumPy)

a^(2): 1 × 1 (scalar), shape (1,) (NumPy)

----- Output Values

Pre-activation a^(2): 0.3922

Network prediction y_hat: 0.3922

True value y: 1.4000

Prediction error: -1.0078

This result indicates that our predicted pedal length and the total result of our pass through the neural network was 0.3922cm, computed from the outputs of our hidden layer.

2.5. Loss Computation

The original y is our observed *true* result, and $\hat{y}$ is the prediction. Our prediction error is $\hat{y} - y = -1.0078$. Our squared error nets us a positive value that more largely penalizes outliers and makes sure the function is differentiable. We may then say our total loss is $\frac{1}{2}$ of that to simplify backpropagation, i.e. $E = \frac{1}{2}(\hat{y} - y)^2$.

Loss Computation

Prediction error:

y is the true length of the petal from the plant providing the other inputs. The prediction error is

$$\hat{y} - y = 0.3922 - 1.4 = -1.0078$$

Squared error:

We can square this to set the ~~error~~ error to positive, & more largely penalize outliers, & ~~make~~ make sure the function is differentiable.

$$(\hat{y} - y)^2 = (-1.0078)^2 \approx 1.0157$$

Loss:

$$E = \frac{1}{2}(1.0157) = 0.5078$$

We may use this to simplify our gradient $\frac{\partial E}{\partial y} = \frac{1}{2} \cdot 2(\hat{y} - y) = \hat{y} - y$ for easier backpropagation operations. For simplicity, we will use $E = \frac{1}{2}(1.0157)$ instead of the computed value

Notebook Cell 8 Output:

Mean Squared Error (MSE) Loss:

$$E = (1/2) * (y_hat - y)^2$$

Step-by-Step Calculation

1. Prediction error: $y_hat - y = 0.3922 - 1.4000 = -1.0078$
2. Squared error: $(-1.0078)^2 = 1.0156$
3. Loss: $E = (1/2) * 1.0156 = 0.5078$

Using our loss data, we may move over to **Backpropagation** to observe how greatly certain values within our weights affect the error and minimize them.

2.6. Verification Table

Quantity	Manual Calculation	Cell Output	Match
$a_1^{[1]}$	-0.223	-0.2229	✓*
$a_2^{[1]}$	-2.0713	-2.0714	✓
$a_3^{[1]}$	-0.8225	-0.8226	✓
$a_4^{[1]}$	-0.9679	-0.9679	✓
$a_5^{[1]}$	2.9712	2.9711	✓
$z_1^{[1]}$	-0.2194	-0.2193	✓
$z_2^{[1]}$	-0.9687	-0.9687	✓
$z_3^{[1]}$	-0.6765	-0.6765	✓
$z_4^{[1]}$	-0.7478	-0.7478	✓
$z_5^{[1]}$	0.9948	0.9948	✓
$\hat{y}$	0.3922	0.3922	✓
$(\hat{y} - y)$	-1.0078	-1.0078	✓
$(\hat{y} - y)$	1.0157	1.0156	✓
E	0.5078	0.5078	✓

Checkpoint 3.

3.1. Output Layer Gradient Derivation

We begin with finding the gradients using partial derivatives in the inverse order of the neural network, beginning with $\frac{\partial E}{\partial a^{[2]}}$.

Handwritten notes on a spiral-bound notebook showing the derivation of the output layer gradient:

Note: Loss $E = \frac{1}{2}(a^{[2]} - y)^2 = \frac{1}{2}(\hat{y} - y)^2$

~~$E = \frac{1}{2}u^2$~~ let $u = \hat{y} - y$

$\hookrightarrow E = \frac{1}{2}u^2$

Chain Rule $\frac{\partial E}{\partial a^{[2]}} = \frac{\partial E}{\partial u} \cdot \frac{\partial u}{\partial a^{[2]}} = \frac{1}{2} \cdot 2u \cdot 1 = u = \hat{y} - y$

$\hookrightarrow \frac{\partial E}{\partial u} = E' = u, \frac{\partial u}{\partial a^{[2]}} = 1, \text{ since } (\hat{y})' = 1$

$\hookrightarrow \frac{\partial E}{\partial a^{[2]}} = \hat{y} - y = -1.0078$ ← from checkpoint

Notebook Cell 9 Output:

```
-----
Derivation: dE/da^(2) = d/da^(2) [(1/2)(a^(2) - y)^2]
              = (1/2) * 2 * (a^(2) - y) * 1
              = a^(2) - y

Result: dE/da^(2) = -1.0078

-----
Error Signal at Output
-----
delta^(2) = dE/dz^(2) = dE/da^(2) (output activation is linear)

Result: delta^(2) = -1.0078
```

Considering this output is negative and relatively far from the expected value, our output is too low. We should be looking for an output as close to 0 as we can.

3.2. Gradient for $W^{[2]}$

$$\frac{\partial E}{\partial W^{[2]}} \Rightarrow$$

for $W_{ij}^{[2]}$ in $W^{[2]}$,
$$\frac{\partial E}{\partial W_{ij}^{[2]}} = (\hat{y} - y) \cdot z_j^{[1]}$$

Since $(\hat{y} - y)$ is constant, we can do scalar multiplication for $\frac{\partial E}{\partial W^{[2]}}$

$$\Rightarrow \frac{\partial E}{\partial W^{[2]}} = (\hat{y} - y) (z^{[1]})^T, \quad z^{[1]T} \in \mathbb{R}^{1 \times 5}$$

$$\Rightarrow \frac{\partial E}{\partial W^{[2]}} = -1.0078 \begin{bmatrix} -0.2194, -0.9687, -0.6765, -0.7478, 0.9948 \end{bmatrix}$$

$$= \begin{bmatrix} 0.2211, 0.9763, 0.6818, 0.7536, -1.0026 \end{bmatrix}$$

Notebook Cell 9 Output

Gradient w.r.t. $W^{(2)}$

Derivation: $dE/dW^{(2)} = (y_hat - y) * (z^{(1)})^T$

NumPy shape: (1, 5)

Mathematical shape: 1×5

Should match $W^{(2)}$: (1, 5) ✓

Values (rounded to 4 decimals):

$\begin{bmatrix} 0.221 & 0.9763 & 0.6818 & 0.7536 & -1.0025 \end{bmatrix}$

3.3. Gradient for $b^{[2]}$

Gradient of $b^{[2]}$

given that $\frac{\partial E}{\partial b^{[2]}} = \hat{y} - y$

$\frac{\partial E}{\partial b^{[2]}} = -1.0078$

Notebook Cell 9 Output:

```
-----  
Gradient w.r.t.  $b^{(2)}$   
-----  
Derivation:  $dE/db^{(2)} = dE/da^{(2)} = y\_hat - y$   
Result:  $dE/db^{(2)} = -1.0078$ 
```

3.4 Error Signal at Hidden Layer

We've now reached the hidden layer in our backpropagation. Here we can observe how the specific randomized weights have affected our output loss E by solving for $\delta^{[1]} = \frac{\partial E}{\partial z^{[1]}}$.

back through $a^{[2]} = W^{[2]} z^{[1]} + b^{[2]}$

$\delta^{[1]} = \frac{\partial E}{\partial z^{[1]}} = \frac{\partial E}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[1]}}$

↑
Computed Previously
as $\hat{y} - y$

Given $\frac{\partial a^{[2]}}{\partial z^{[1]}} = W^{[2]}$, $W^{[2]}$ being a row vector $\in \mathbb{R}^{1 \times 5}$,

$\delta^{[1]} = (W^{[2]})^T (\hat{y} - y)$, $\delta^{[1]} \in \mathbb{R}^{5 \times 1}$

$$\rightarrow \delta^{[1]} = (w^{[2]})^T \cdot (\hat{y} - y) = (-1.0078)(w^{[2]})^T, \delta^{[1]} \in \mathbb{R}^{5 \times 1}$$

$$\Rightarrow \begin{aligned} \delta_1^{[1]} &= (-1.0078)(-0.2241) = 0.2258 \\ \delta_2^{[1]} &= (-1.0078)(0.0812) = -0.0818 \\ \delta_3^{[1]} &= (-1.0078)(0.1286) = -0.1296 \\ \delta_4^{[1]} &= (-1.0078)(-0.6377) = 0.6427 \\ \delta_5^{[1]} &= (-1.0078)(0.032) = -0.0322 \end{aligned}$$

$$\Rightarrow \delta^{[1]} = \begin{bmatrix} 0.2258 \\ -0.0818 \\ -0.1296 \\ 0.6427 \\ -0.0322 \end{bmatrix}$$

Notebook Cell 10 Output:

```
-----
delta^(1) = dE/dz^(1) Values
```

```
-----
Values (rounded to 4 decimals):
```

```
[ 0.2258 -0.0818 -0.1296  0.6427 -0.0323]
```

The values within this vector directly correlate to how much each neuron in the same slot contributes to the error, with numbers with larger absolute values having provided a much stronger effect. This is to say that the larger $|\delta_i^{[1]}|$ is, the more $z_i^{[1]}$ affects E . These values require greater adjustment to reduce our error.

3.5. Activation Function Derivative

Next, we find $h'(a^{[1]})$:

Activation Function Derivative

Recall $h(z) = \tanh(z) \Rightarrow$

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad \text{let } e^z - e^{-z} = u, \quad e^z + e^{-z} = v$$

$$\frac{d}{dz}\left(\frac{u}{v}\right) = \frac{u'v - uv'}{v^2}$$

$$\hookrightarrow \frac{(e^z - e^{-z})'(e^z + e^{-z}) - (e^z - e^{-z})(e^z + e^{-z})'}{(e^z + e^{-z})^2}$$

$$(e^z)' = e^z$$

$$(e^{-z})' = -e^{-z}$$

$$\hookrightarrow \frac{(e^z + e^{-z})(e^z + e^{-z}) - (e^z - e^{-z})(e^z - e^{-z})}{(e^z + e^{-z})^2}$$

$$= \frac{e^{2z} + 2 + e^{-2z} - [e^{2z} - 2 + e^{-2z}]}{(e^z + e^{-z})^2} = \frac{4}{(e^z + e^{-z})^2}$$

$$= \frac{4}{(e^z + e^{-z})^2} = 1 - \tanh^2(z)$$

Recall $z^{[1]} = \begin{bmatrix} -0.2194 \\ -0.9687 \\ -0.6765 \\ -0.7478 \\ 0.9948 \end{bmatrix}, \quad z_i^{[1]} = \tanh(a_i)$

$\hookrightarrow h'(a_i^{[1]}) = 1 - (z_i^{[1]})^2$

$$\hookrightarrow \begin{aligned} h'(a_1^{[1]}) &= 1 - (-0.2194)^2 = 0.9519 \\ h'(a_2^{[1]}) &= 1 - (-0.9687)^2 = 0.0616 \\ h'(a_3^{[1]}) &= 1 - (-0.6765)^2 = 0.5423 \\ h'(a_4^{[1]}) &= 1 - (-0.7478)^2 = 0.4408 \\ h'(a_5^{[1]}) &= 1 - (0.9948)^2 = 0.0104 \end{aligned}$$

$\Rightarrow h'(a^{[1]}) = \begin{bmatrix} 0.9519 \\ 0.0616 \\ 0.5423 \\ 0.4408 \\ 0.0104 \end{bmatrix}$

Notebook Cell 11 Output:

Activation Function Derivative

For tanh: $h'(a) = 1 - \tanh^2(a) = 1 - z^2$
(Derived using quotient rule in Checkpoint 3)

$h'(a^{(1)})$ values (rounded to 4 decimals):
[0.9519 0.0615 0.5423 0.4408 0.0105]

3.6. Gradient With Respect to Pre-Activations

Gradient With Respect to Pre-Activations $\Rightarrow \frac{\partial E}{\partial a^{[1]}}$

Given $\frac{\partial E}{\partial a_i^{[1]}} = \frac{\partial E}{\partial z_i^{[1]}} \cdot \frac{\partial z_i^{[1]}}{\partial a_i^{[1]}} = \delta_i^{[1]} \cdot h'(a_i^{[1]})$,

we can say $\frac{\partial E}{\partial a^{[1]}} = \delta^{[1]} \odot h'(a^{[1]})$

$$= \begin{bmatrix} 0.2258 * 0.9519 \\ -0.0818 * 0.0616 \\ -0.1296 * 0.5423 \\ 0.6427 * 0.4408 \\ -0.0322 * 0.0104 \end{bmatrix} = \begin{bmatrix} 0.2149 \\ -0.005 \\ -0.0703 \\ 0.2833 \\ -0.0003 \end{bmatrix} = \frac{\partial E}{\partial a^{[1]}}$$

Notebook Cell 11 Output:

Dimensional Analysis

$\delta^{(1)}$: 5 × 1 (column vector)
 $h'(a^{(1)})$: 5 × 1 (column vector)

Result: 5 × 1 (column vector)

$dE/da^{(1)}$ Values

Values (rounded to 4 decimals):
[0.2149 -0.005 -0.0703 0.2833 -0.0003]

Since we are using **Hadamard Product**, denoted by $\odot$, we simply multiply elementwise rather than utilizing matrix multiplication. This product continues the process of backpropagation and observing how much error goes back through each neuron.

3.7. Gradient for $W^{[1]}$

Gradient for $W^{[1]}$ $\frac{\partial E}{\partial W^{[1]}}$ $Recall\ x = \begin{bmatrix} 5.1 \\ 3.5 \\ 0.2 \end{bmatrix}$

Given $\frac{\partial E}{\partial w_{ij}^{[1]}} = \frac{\partial E}{\partial a_i^{[1]}} \cdot \frac{\partial a_i^{[1]}}{\partial w_{ij}^{[1]}}$ & $\frac{\partial a_i^{[1]}}{\partial w_{ij}^{[1]}} = x_j$,

$\Rightarrow \frac{\partial E}{\partial w_{ij}^{[1]}} = \frac{\partial E}{\partial a_i^{[1]}} x_j \Rightarrow \frac{\partial E}{\partial W^{[1]}} = \frac{\partial E}{\partial a^{[1]}} x^T$

$=_{5 \times 1} \begin{bmatrix} 0.2149 \\ -0.005 \\ -0.0703 \\ 0.2833 \\ -0.0003 \end{bmatrix} [5.1, 3.5, 0.2]^{1 \times 3}$

$=_{5 \times 3} \begin{bmatrix} (0.2149)(5.1) & (0.2149)(3.5) & (0.2149)(0.2) \\ (-0.005)(5.1) & (-0.005)(3.5) & (-0.005)(0.2) \\ (-0.0703)(5.1) & (-0.0703)(3.5) & (-0.0703)(0.2) \\ (0.2833)(5.1) & (0.2833)(3.5) & (0.2833)(0.2) \\ (-0.0003)(5.1) & (-0.0003)(3.5) & (-0.0003)(0.2) \end{bmatrix}$

$= \begin{bmatrix} 1.096 & 0.7522 & 0.043 \\ -0.0255 & -0.0175 & -0.001 \\ -0.3585 & -0.2461 & -0.0141 \\ 1.4448 & 0.9916 & 0.0567 \\ -0.0015 & -0.0011 & -0.0001 \end{bmatrix} = \frac{\partial E}{\partial W^{[1]}}$

Notebook Cell 11 Output:

```
Derivation: dE/dW^(1) = outer_product(dE/da^(1), x)
Mathematical: (5 x 1) ⊗ (1 x 3) = (5 x 3)

NumPy shape:      (5, 3)
Should match W^(1): (5, 3) ✓

Values (rounded to 4 decimals):
[[ 1.0962e+00  7.5230e-01  4.3000e-02]
 [-2.5700e-02 -1.7600e-02 -1.0000e-03]
 [-3.5850e-01 -2.4600e-01 -1.4100e-02]
 [ 1.4448e+00  9.9160e-01  5.6700e-02]
 [-1.7000e-03 -1.2000e-03 -1.0000e-04]]
```

3.8. Gradient for $b^{[1]}$

Gradient for $b^{[1]} \Rightarrow \frac{\partial E}{\partial b^{[1]}}$

Given $\frac{\partial E}{\partial b^{[1]}} = \frac{\partial E}{\partial a^{[1]}} \cdot \frac{\partial a^{[1]}}{\partial b^{[1]}} = \frac{\partial E}{\partial a^{[1]}} \cdot 1$

$\frac{\partial E}{\partial b^{[1]}} = \frac{\partial E}{\partial a^{[1]}} = \begin{bmatrix} 0.2149 \\ -0.005 \\ -0.0703 \\ 0.2833 \\ -0.0003 \end{bmatrix}$

Notebook Cell 11 Output:

```
-----
Gradient w.r.t. b^(1)
-----
Derivation: dE/db^(1) = dE/da^(1)
Shape: (5,)

Values (rounded to 4 decimals):
[ 0.2149 -0.005 -0.0703  0.2833 -0.0003]
```

This final gradient tells us how much each weight should be adjusted to minimize loss.

3.9. Complete Chain Rule Example

For my example run-through of the chain rule, I moved over to $W_{13}^{[1]}$. The following two python snippets are the modifications of the original code to test this specific value

The python code initializing the factors and product and comparing to the same position in the gradient for $W^{[1]}$:

```
factor1 = dE_da2
factor2 = W2[0, 0] # w_{11}^(2)
factor3 = h_prime_a1[0] # h'(a_1^(1))
factor4 = x[2] # x_2 = 3.5
chain_product = factor1 * factor2 * factor3 * factor4

print(f" Match: {np.isclose(chain_product, dE_dw1[0, 2])} ✓ if np.isclose(chain_product, dE_dw1[0, 2]) else f" Match: False X")
```

Now we compute $\frac{\partial E}{\partial w_{13}^{[1]}} = \frac{\partial E}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z_1^{[1]}} \cdot \frac{\partial z_1^{[1]}}{\partial a_1^{[1]}} \cdot \frac{\partial a_1^{[1]}}{\partial w_{13}^{[1]}}$.

~~Chain~~ Complete Chain Rule Example

Let's pick $w_{13}^{[1]} = 0.5209$

$$\frac{\partial E}{\partial w_{13}^{[1]}} = \frac{\partial E}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z_1^{[1]}} \cdot \frac{\partial z_1^{[1]}}{\partial a_1^{[1]}} \cdot \frac{\partial a_1^{[1]}}{\partial w_{13}^{[1]}}$$

$$\frac{\partial a_1^{[1]}}{\partial w_{13}^{[1]}} = x_j \Rightarrow \frac{\partial a_1^{[1]}}{\partial w_{13}^{[1]}} = x_3 = 0.2$$

$$\frac{\partial z_1^{[1]}}{\partial a_1^{[1]}} = \text{~~h'(a_1^{[1]})~~} \Rightarrow \frac{\partial z_1^{[1]}}{\partial a_1^{[1]}} = \text{~~0.9519~~} \cdot 0.9519$$

$$\frac{\partial a^{[2]}}{\partial z_1^{[1]}} = w^{[2]} \Rightarrow \frac{\partial a^{[2]}}{\partial z_1^{[1]}} = w_1^{[2]} = -0.2241$$

$$\frac{\partial E}{\partial a^{[2]}} = -1.0078$$

$$\Rightarrow \frac{\partial E}{\partial w_{13}^{[1]}} = (0.2)(0.9519)(-0.2241)(-1.0078) = 0.043$$

Notebook Cell 12 Output:

```
Full chain:
dE/dw_{12}^{(1)} = dE/da^{(2)} * da^{(2)}/dz_1^{(1)} * dz_1^{(1)}/da_1^{(1)} * da_1^{(1)}/dw_{12}^{(1)}

Factor 1: dE/da^{(2)}           = -1.0078
Factor 2: da^{(2)}/dz_1^{(1)}    = w_{11}^{(2)} = -0.2241
Factor 3: dz_1^{(1)}/da_1^{(1)}   = h'(a_1^{(1)}) = 0.9519
Factor 4: da_1^{(1)}/dw_{12}^{(1)} = x_2 = 0.2000

Chain rule product:           = 0.0430
From gradient matrix dE/dW^{(1)}[0,1]: 0.0430
Match: True ✓
```

3.10. Verification Table

Quantity	Manual Calculation	Cell Output	Match
$\frac{\partial E}{\partial \mathbf{a}^{[2]}}$	-1.0078	-1.0078	✓
$\frac{\partial E}{\partial \mathbf{w}_1^{[2]}}$	0.2211	0.221	✓
$\frac{\partial E}{\partial \mathbf{w}_2^{[2]}}$	0.9763	0.9763	✓
$\frac{\partial E}{\partial \mathbf{w}_3^{[2]}}$	0.6818	0.6818	✓
$\frac{\partial E}{\partial \mathbf{w}_4^{[2]}}$	0.7536	0.7536	✓
$\frac{\partial E}{\partial \mathbf{b}^{[2]}}$	-1.0078	-1.0078	✓
$\frac{\partial E}{\partial \mathbf{w}_5^{[2]}}$	-1.0026	-1.0025	✓
$\delta_1^{[1]}$	0.2258	0.2258	✓
$\delta_2^{[1]}$	-0.0818	-0.0818	✓
$\delta_3^{[1]}$	-0.1296	-0.1296	✓
$\delta_4^{[1]}$	0.6427	0.6427	✓
$\delta_5^{[1]}$	-0.0322	-0.0323	✓
$h'(\mathbf{a}_1^{[1]})$	0.9519	0.9519	✓
$h'(\mathbf{a}_2^{[1]})$	0.0616	0.0615	✓
$h'(\mathbf{a}_3^{[1]})$	0.5423	0.5423	✓
$h'(\mathbf{a}_4^{[1]})$	0.4408	0.4408	✓
$h'(\mathbf{a}_5^{[1]})$	0.0104	0.0105	✓
$\frac{\partial E}{\partial \mathbf{a}_1^{[1]}}$	0.2149	0.2149	✓
$\frac{\partial E}{\partial \mathbf{a}_2^{[1]}}$	-0.005	-0.005	✓
$\frac{\partial E}{\partial \mathbf{a}_3^{[1]}}$	-0.0703	-0.0703	✓
$\frac{\partial E}{\partial \mathbf{a}_4^{[1]}}$	0.2833	0.2833	✓
$\frac{\partial E}{\partial \mathbf{a}_5^{[1]}}$	-0.0003	-0.0003	✓

$\frac{\partial E}{\partial \mathbf{w}_{11}^{[1]}}$	1.096	1.0962	✓
$\frac{\partial E}{\partial \mathbf{w}_{12}^{[1]}}$	0.7522	0.7523	✓
$\frac{\partial E}{\partial \mathbf{w}_{13}^{[1]}}$	0.043	0.043	✓
$\frac{\partial E}{\partial \mathbf{w}_{21}^{[1]}}$	-0.0255	-0.0257	✓
$\frac{\partial E}{\partial \mathbf{w}_{22}^{[1]}}$	-0.0175	-0.0176	✓
$\frac{\partial E}{\partial \mathbf{w}_{23}^{[1]}}$	-0.001	-0.001	✓
$\frac{\partial E}{\partial \mathbf{w}_{31}^{[1]}}$	-0.3585	-0.3585	✓
$\frac{\partial E}{\partial \mathbf{w}_{32}^{[1]}}$	-0.2461	-0.246	✓
$\frac{\partial E}{\partial \mathbf{w}_{33}^{[1]}}$	-0.0141	-0.0141	✓
$\frac{\partial E}{\partial \mathbf{w}_{41}^{[1]}}$	1.4448	1.4448	✓
$\frac{\partial E}{\partial \mathbf{w}_{42}^{[1]}}$	0.9916	0.9916	✓
$\frac{\partial E}{\partial \mathbf{w}_{43}^{[1]}}$	0.0567	0.0567	✓
$\frac{\partial E}{\partial \mathbf{w}_{51}^{[1]}}$	-0.0015	-0.0017	✓
$\frac{\partial E}{\partial \mathbf{w}_{52}^{[1]}}$	-0.0011	-0.0012	✓
$\frac{\partial E}{\partial \mathbf{w}_{53}^{[1]}}$	-0.0001	-0.0001	✓
$\frac{\partial E}{\partial \mathbf{b}_1^{[1]}} \quad \frac{\partial E}{\partial \mathbf{a}_1^{[1]}}$	0.2149	0.2149	✓
$\frac{\partial E}{\partial \mathbf{b}_1^{[1]}} \quad \frac{\partial E}{\partial \mathbf{a}_1^{[1]}}$	-0.005	-0.005	✓
$\frac{\partial E}{\partial \mathbf{b}_1^{[1]}} \quad \frac{\partial E}{\partial \mathbf{a}_1^{[1]}}$	-0.0703	-0.0703	✓
$\frac{\partial E}{\partial \mathbf{b}_1^{[1]}} \quad \frac{\partial E}{\partial \mathbf{a}_1^{[1]}}$	0.2833	0.2833	✓
$\frac{\partial E}{\partial \mathbf{b}_1^{[1]}} \quad \frac{\partial E}{\partial \mathbf{a}_1^{[1]}}$	-0.0003	-0.0003	✓

Notebook Summary Screenshot(s)

```
=====
COMPLETE NEURAL NETWORK COMPUTATION SUMMARY
=====

#####
# CONFIGURATION
#####

Random seed:      21
Network architecture: 3 → 5 → 1
Activation function: tanh (or ReLU if modified)
Loss function:    MSE with factor 1/2

#####
# INPUT DATA (First data point from Iris)
#####

x (features): [5.1 3.5 0.2]
  • Sepal length: 5.1
  • Sepal width: 3.5
  • Petal width: 0.2

y (target petal length): 1.4000

=====
FORWARD PASS RESULTS (for Checkpoint 2)
=====

-----
Hidden Layer Pre-Activations a^(1)
-----

Mathematical shape: 5 × 1
NumPy shape:      (5,)

Values:
a_1^(1) = -0.2229
a_2^(1) = -2.0714
a_3^(1) = -0.8226
a_4^(1) = -0.9679
a_5^(1) = 2.9711

-----
Hidden Layer Activations z^(1)
-----

Mathematical shape: 5 × 1
NumPy shape:      (5,)

Values:
z_1^(1) = -0.2193
z_2^(1) = -0.9687
z_3^(1) = -0.6765
z_4^(1) = -0.7478
z_5^(1) = 0.9948

-----
Output Layer
-----

a^(2) (pre-activation): 0.3922
y_hat (prediction):      0.3922
y (true value):          1.4000
Prediction error:        -1.0078

-----
Loss
-----

E = 0.5078

=====
BACKPROPAGATION RESULTS (for Checkpoint 3)
=====

-----
Output Layer Gradients
-----

dE/da^(2): -1.0078
delta^(2): -1.0078

-----
Gradients w.r.t. W^(2)
-----

Shape: (1, 5)

Values:
[[ 0.221  0.9763  0.6818  0.7536 -1.0025]]
```

Gradient w.r.t. b^(2)

dE/db^(2): -1.0078

Hidden Layer Error Signal delta^(1) = dE/dz^(1)

Values:

delta_1^(1) = 0.2258
delta_2^(1) = -0.0818
delta_3^(1) = -0.1296
delta_4^(1) = 0.6427
delta_5^(1) = -0.0323

Activation Derivatives h'(a^(1))

Values:

h'(a_1^(1)) = 0.9519
h'(a_2^(1)) = 0.0615
h'(a_3^(1)) = 0.5423
h'(a_4^(1)) = 0.4408
h'(a_5^(1)) = 0.0105

Pre-Activation Gradients dE/da^(1)

Values:

dE/da_1^(1) = 0.2149
dE/da_2^(1) = -0.0050
dE/da_3^(1) = -0.0703
dE/da_4^(1) = 0.2833
dE/da_5^(1) = -0.0003

Gradients w.r.t. W^(1)

Shape: (5, 3)

Values:

[[1.0962e+00 7.5230e-01 4.3000e-02]
 [-2.5700e-02 -1.7600e-02 -1.0000e-03]
 [-3.5850e-01 -2.4600e-01 -1.4100e-02]
 [1.4448e+00 9.9160e-01 5.6700e-02]
 [-1.7000e-03 -1.2000e-03 -1.0000e-04]]

Gradients w.r.t. b^(1)

Values:

dE/db_1^(1) = 0.2149
dE/db_2^(1) = -0.0050
dE/db_3^(1) = -0.0703
dE/db_4^(1) = 0.2833
dE/db_5^(1) = -0.0003

Appendices

Appendix A. Handwritten Arithmetic for Forward Pass

Checkpoint 2

First data point: $x = \begin{bmatrix} 5.1 \\ 3.5 \\ 0.2 \end{bmatrix}$ $y = 1.4$

* $W^{[1]}$ for checkpoint 1

$a^{[1]} = W^{[1]}x =$

Weight Data point 1

$a^{[1]} = \begin{bmatrix} -0.026 & -0.0556 & 0.5209 \\ -0.6214 & 0.3727 & -0.8555 \\ -0.1023 & -0.1173 & 0.5641 \\ -0.0065 & -0.3066 & 0.6818 \\ -0.8855 & -0.3446 & 0.346 \end{bmatrix} \begin{bmatrix} 5.1 \\ 3.5 \\ 0.2 \end{bmatrix}$

* Input note:

Coming back to check, some values only make 2 decimal places. Likely due to rounding rules, but arithmetic was valid.

$a^{[1]} = \begin{bmatrix} -0.1526 & 0.1746 & 0.10415 \\ -3.20464 & 1.30445 & -0.1711 \\ -0.52473 & -0.41955 & 0.11132 \\ -0.03213 & -1.0751 & 0.15736 \\ 4.10805 & -1.2061 & 0.0692 \end{bmatrix} \begin{bmatrix} 5.1 \\ 3.5 \\ 0.2 \end{bmatrix} \approx \begin{bmatrix} -0.223 \\ -2.0715 \\ -0.8215 \\ -0.7679 \\ 2.9712 \end{bmatrix}$

Carrying over $\Rightarrow$

$a^{[1]} = \begin{bmatrix} -0.223 \\ -2.0715 \\ -0.8215 \\ -0.7679 \\ 2.9712 \end{bmatrix}^T$

This is a row vector (1×5) $\Rightarrow$ input distribution

$a^{[1]}$ is our activation vector after our input vector/neuron/layer

* Note: $a^{[1]} \cdot z^{[1]} \in \mathbb{R}^{1 \times 5}$ Our selected activation function

Hidden Layer Activation ($a^{[1]} \Rightarrow z^{[1]}$)

for all $z_i^{[1]}, z_1^{[1]}, z_2^{[1]}, z_3^{[1]}, z_4^{[1]}, z_5^{[1]} = \tanh(a_i^{[1]})$

$z_1^{[1]} = \frac{e^{a_1^{[1]}} - e^{-a_1^{[1]}}}{e^{a_1^{[1]}} + e^{-a_1^{[1]}}} = \frac{e^{-0.223} - e^{0.223}}{e^{-0.223} + e^{0.223}} = -0.2194$

$z_2^{[1]} = \frac{e^{-2.0715} - e^{2.0715}}{e^{-2.0715} + e^{2.0715}} = -0.9687$

$z_3^{[1]} = \frac{e^{-0.8215} - e^{0.8215}}{e^{-0.8215} + e^{0.8215}} = -0.6765$

$z_4^{[1]} = \frac{e^{-0.7679} - e^{0.7679}}{e^{-0.7679} + e^{0.7679}} = -0.7478$

$z_5^{[1]} = \frac{e^{2.9712} - e^{-2.9712}}{e^{2.9712} + e^{-2.9712}} = 0.9948$

$z^{[1]} = \begin{bmatrix} -0.2194 \\ -0.9687 \\ -0.6765 \\ -0.7478 \\ 0.9948 \end{bmatrix}$

Carrying over $\Rightarrow$

$z^{[1]} = \begin{bmatrix} -0.2194 \\ -0.9687 \\ -0.6765 \\ -0.7478 \\ 0.9948 \end{bmatrix}^T$

* Once again, bias is 0

$z^{[1]}$ is the vector containing activated outputs of the hidden neuron vector $a^{[1]}$. The tanh activation function reduces the range of values to $(-1, 1)$ & introduces non-linearity, allowing more complex analysis.

Output Layer Activation $\Rightarrow$ $W^{[2]}z^{[1]} = a^{[2]}, \hat{y} = a^{[2]}$

Since we are moving to the output layer, the activation layer is linear, so $a^{[2]} = z^{[1]} = \hat{y}$

Thus, $W^{[2]}z^{[1]} = \hat{y}$

$= \begin{bmatrix} 0.2241 & 0.0012 & 0.1286 & 0.6377 & 0.032 \end{bmatrix} \begin{bmatrix} -0.2194 \\ -0.9687 \\ -0.6765 \\ -0.7478 \\ 0.9948 \end{bmatrix}$

$= \begin{bmatrix} (-0.2194)(0.2241) + (0.0012)(0.0012) + (0.1286)(0.1286) + (0.6377)(0.6377) + (0.032)(0.032) \\ 0.0092 + 0.0012 - 0.0087 + 0.4067 + 0.0010 \end{bmatrix}$

$= \begin{bmatrix} 0.3922 \end{bmatrix}$

$\Rightarrow a^{[2]} = \begin{bmatrix} 0.3922 \end{bmatrix} \Rightarrow \hat{y} = 0.3922$

Carrying over $\Rightarrow$ $\hat{y} = 0.3922, y = 1.4$

$\hat{y} = 0.3922$. This indicates that our current predicted weightless prediction for pedal length is 0.3922cm, computed from our hidden layer neuron outputs.

Loss Computation

Prediction error:

y is the true length of the pedal from the plant providing the other inputs. The prediction error is $\hat{y} - y = 0.3922 - 1.4 = -1.0078$

Squared error:

We can square this to get the number error to positive & more largely penalize outliers, & make sure the function is differentiable.

$(\hat{y} - y)^2 = (-1.0078)^2 \approx 1.0157$

Carrying over $\Rightarrow (\hat{y} - y)^2 = 1.0157$

Loss:

$E = \frac{1}{2}(1.0157) = 0.5078$

Appendix B. Handwritten Derivative Work for Backpropagation

Checkpoint 3
Output Layer Gradient derivation

Note: Loss $E = \frac{1}{2}(a^{[2]} - y)^2 = \frac{1}{2}(\hat{y} - y)^2$

Let $u = \hat{y} - y$

$E = \frac{1}{2}u^2$

Chain Rule $\frac{\partial E}{\partial a^{[2]}} = \frac{\partial E}{\partial u} \cdot \frac{\partial u}{\partial a^{[2]}} = \frac{1}{2} \cdot 2u \cdot 1 = u = \hat{y} - y$

$\frac{\partial E}{\partial u} = E' = u$, $\frac{\partial u}{\partial a^{[2]}} = 1$ (since $\hat{y} = a^{[2]}$)

$\frac{\partial E}{\partial a^{[2]}} = \hat{y} - y = -1.0078$ (from checkpoint 1)

Gradient of $b^{[2]}$

given that $\frac{\partial E}{\partial a^{[2]}} = \hat{y} - y$

$\frac{\partial E}{\partial b^{[2]}} = -1.0078$

Error Signal at Hidden Layer $\delta^{[1]} = \frac{\partial E}{\partial z^{[1]}}$

This is entering our backpropagation

back through $a^{[2]} = W^{[2]} z^{[1]} + b^{[2]}$

$\delta^{[1]} = \frac{\partial E}{\partial z^{[1]}} = \frac{\partial E}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[1]}}$

Computed previously as $\hat{y} - y$

Given $\frac{\partial a^{[2]}}{\partial z^{[1]}} = W^{[2]}$, $W^{[2]}$ being a row vector $\in \mathbb{R}^{1 \times 5}$

$\delta^{[1]} = (W^{[2]})^T (\hat{y} - y)$, $\delta^{[1]} \in \mathbb{R}^{5 \times 1}$

Arithmetic Next Page

Activation Function Derivative

Recall $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$

Let $u = e^z - e^{-z}$, $v = e^z + e^{-z}$

$\frac{d}{dz} \left(\frac{u}{v} \right) = \frac{u'v - uv'}{v^2}$

$u' = e^z + e^{-z} = v$, $v' = e^z - e^{-z} = u$

$\frac{d}{dz} \left(\frac{u}{v} \right) = \frac{v^2 - uv}{v^2} = \frac{v^2 - u^2}{v^2}$

$\frac{d}{dz} \left(\frac{e^z - e^{-z}}{e^z + e^{-z}} \right) = \frac{(e^z + e^{-z})^2 - (e^z - e^{-z})^2}{(e^z + e^{-z})^2}$

$= \frac{e^{2z} + 2 + e^{-2z} - (e^{2z} - 2 + e^{-2z})}{(e^z + e^{-z})^2} = \frac{4}{(e^z + e^{-z})^2}$

$= \frac{4}{(e^z + e^{-z})^2} = 1 - \tanh^2(z)$

finding all $h'(a^{[1]})$ next page

Gradient for $W^{[2]}$

$\frac{\partial E}{\partial W^{[2]}} \Rightarrow$

for $W_{ij}^{[2]} = a_i^{[2]} \cdot W_{ij}^{[2]} = (\hat{y} - y) \cdot z_j^{[1]}$

Since $(\hat{y} - y)$ is constant, we can do scalar multiplication for $\frac{\partial E}{\partial W^{[2]}}$

$\frac{\partial E}{\partial W^{[2]}} = (\hat{y} - y) (z^{[1]})^T$, $z^{[1]T} \in \mathbb{R}^{1 \times 5}$

$\Rightarrow \frac{\partial E}{\partial W^{[2]}} = -1.0078 \begin{bmatrix} 0.2194 & -0.9687 & -0.6765 & -0.7978 & 0.9948 \end{bmatrix}$

$= \begin{bmatrix} 0.2211 & 0.9763 & 0.6818 & 0.7536 & -1.0026 \end{bmatrix}$

$\Rightarrow \delta^{[1]} = (W^{[2]})^T \cdot (\hat{y} - y) = (-1.0078) (W^{[2]})^T \delta^{[2]} \in \mathbb{R}^{5 \times 1}$

$\Rightarrow \delta^{[1]} = \begin{bmatrix} 0.2258 \\ -0.0818 \\ -0.1296 \\ 0.6427 \\ -0.0322 \end{bmatrix}$

The values within this vector tell us how much each neuron in the hidden layer contributed to error. In other words, these we observe which neurons were more influential by how large/small they are, with $\delta^{[1]}$ showing us that it has a larger influence on the error & needs greater adjustment.

Recall $z^{[1]} = \begin{bmatrix} -0.2194 \\ -0.9687 \\ -0.6765 \\ -0.7978 \\ 0.9948 \end{bmatrix}$, $\phi(z^{[1]}) = \tanh(z^{[1]})$

$h'(a_i^{[1]}) = 1 - (z_i^{[1]})^2$

$\hookrightarrow h'(a_1^{[1]}) = 1 - (-0.2194)^2 = 0.9519$
 $h'(a_2^{[1]}) = 1 - (-0.9687)^2 = 0.0616$
 $h'(a_3^{[1]}) = 1 - (-0.6765)^2 = 0.5423$
 $h'(a_4^{[1]}) = 1 - (-0.7978)^2 = 0.4408$
 $h'(a_5^{[1]}) = 1 - (0.9948)^2 = 0.0109$

$\Rightarrow h'(a^{[1]}) = \begin{bmatrix} 0.9519 \\ 0.0616 \\ 0.5423 \\ 0.4408 \\ 0.0109 \end{bmatrix}$

Gradient With Respect to Pre-Activations $\Rightarrow \frac{\partial E}{\partial a_i^{[1]}}$

Given $\frac{\partial E}{\partial a_i^{[1]}} = \frac{\partial E}{\partial z_i^{[1]}} \cdot \frac{\partial z_i^{[1]}}{\partial a_i^{[1]}} = \delta_i^{[1]} \cdot h'(a_i^{[1]})$

We can say $\frac{\partial E}{\partial a_i^{[1]}} = \delta_i^{[1]} \odot h'(a_i^{[1]})$

$$= \begin{bmatrix} 0.2258 & 0.9519 \\ -0.0818 & 0.0616 \\ -0.1296 & 0.5423 \\ 0.6427 & 0.4408 \\ 0.0322 & 0.0104 \end{bmatrix} \cdot \begin{bmatrix} 0.2149 \\ -0.005 \\ -0.703 \\ 0.2833 \\ -0.0003 \end{bmatrix} = \frac{\partial E}{\partial a_i^{[1]}}$$

We are continuing the process of carrying back error weights. We are firstly scaling across matrices, similar to a scalar operation

Gradient for $w_{13}^{[1]} \Rightarrow \frac{\partial E}{\partial w_{13}^{[1]}}$ Recall $x = \begin{bmatrix} 5.1 \\ 3.5 \\ 0.2 \end{bmatrix}$

Given $\frac{\partial E}{\partial w_{13}^{[1]}} = \frac{\partial E}{\partial a_1^{[1]}} \cdot \frac{\partial a_1^{[1]}}{\partial w_{13}^{[1]}} = \frac{\partial a_1^{[1]}}{\partial w_{13}^{[1]}} = x_1$

$\Rightarrow \frac{\partial E}{\partial w_{13}^{[1]}} = \frac{\partial E}{\partial a_1^{[1]}} x_1 \Rightarrow \frac{\partial E}{\partial w_{13}^{[1]}} = \frac{\partial E}{\partial a_1^{[1]}} \times T$

$$= \begin{bmatrix} 0.2149 \\ -0.005 \\ -0.703 \\ 0.2833 \\ -0.0003 \end{bmatrix} \begin{bmatrix} 5.1 & 3.5 & 0.2 \end{bmatrix}^{1,3}$$

$$= \begin{bmatrix} (0.2149)(5.1) & (0.2149)(3.5) & (0.2149)(0.2) \\ (-0.005)(5.1) & (-0.005)(3.5) & (-0.005)(0.2) \\ (-0.703)(5.1) & (-0.703)(3.5) & (-0.703)(0.2) \\ (0.2833)(5.1) & (0.2833)(3.5) & (0.2833)(0.2) \\ (-0.0003)(5.1) & (-0.0003)(3.5) & (-0.0003)(0.2) \end{bmatrix}$$

$$= \begin{bmatrix} 1.096 & 0.7522 & 0.043 \\ -0.0255 & -0.0175 & -0.001 \\ -3.5853 & -2.4605 & -0.1406 \\ 1.4448 & 0.9916 & 0.0567 \\ -0.0015 & -0.0011 & -0.0001 \end{bmatrix} = \frac{\partial E}{\partial w_{13}^{[1]}}$$

Gradient for $b^{[1]} \Rightarrow \frac{\partial E}{\partial b^{[1]}}$

Given $\frac{\partial E}{\partial b_i^{[1]}} = \frac{\partial E}{\partial a_i^{[1]}} \cdot \frac{\partial a_i^{[1]}}{\partial b_i^{[1]}} = \frac{\partial E}{\partial a_i^{[1]}} \cdot 1$

$$\frac{\partial E}{\partial b_i^{[1]}} = \frac{\partial E}{\partial a_i^{[1]}} = \begin{bmatrix} 0.2149 \\ -0.005 \\ -0.703 \\ 0.2833 \\ -0.0003 \end{bmatrix}$$

Note: While it was corrected in most of the paper, there was a missing zero in $\frac{\partial E}{\partial a_3^{[1]}}$ that skewed some of the original handwritten results. That mistake is shown here.

Complete Chain Rule Example

Let's pick $w_{13}^{[1]} = 0.5209$

$$\frac{\partial E}{\partial w_{13}^{[1]}} = \frac{\partial E}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z_1^{[1]}} \cdot \frac{\partial z_1^{[1]}}{\partial a_1^{[1]}} \cdot \frac{\partial a_1^{[1]}}{\partial w_{13}^{[1]}}$$

$$\frac{\partial a_1^{[1]}}{\partial w_{13}^{[1]}} = x_j \Rightarrow \frac{\partial a_1^{[1]}}{\partial w_{13}^{[1]}} = x_3 = 0.2$$

$$\frac{\partial z_1^{[1]}}{\partial a_1^{[1]}} = h'(a_1^{[1]}) \Rightarrow \frac{\partial z_1^{[1]}}{\partial a_1^{[1]}} = 0.9519$$

$$\frac{\partial a^{[2]}}{\partial z_1^{[1]}} = w^{[2]} \Rightarrow \frac{\partial a^{[2]}}{\partial z_1^{[1]}} = w^{[2]}_1 = -0.2241$$

$$\frac{\partial E}{\partial a^{[2]}} = -1.0078$$

$$\Rightarrow \frac{\partial E}{\partial w_{13}^{[1]}} = (0.2)(0.9519)(-0.2241)(-1.0078) = 0.043$$

Appendix C. Complete Notebook Run

Cell 1:

```
=====
RANDOM SEED INITIALIZATION
=====
```

```
Random seed set to: 21
```

```
This seed will generate your unique weight values.
All subsequent calculations depend on this seed.
```

Cell 2:

```
=====
NETWORK ARCHITECTURE
=====
```

```
Input layer size: 3
```

```
Hidden layer size: 5
```

```
Output layer size: 1
```

```
Network structure: 3 → 5 → 1
```

```
-----
Expected Weight Dimensions (Mathematical Notation)
-----
```

```
W(1): 5 × 3 (input to hidden weights)
```

```
b(1): 5 × 1 (hidden layer bias)
```

```
W(2): 1 × 5 (hidden to output weights)
```

```
b(2): 1 × 1 (output layer bias)
```


Cell 3:

```
=====
INITIALIZED WEIGHTS AND BIASES
=====

-----
W^(1): Weights from Input to Hidden Layer
-----
NumPy shape:      (5, 3)
Mathematical shape: 5 × 3

Values (rounded to 4 decimals):
[[-0.026 -0.0556  0.5209]
 [-0.6284  0.3727 -0.8555]
 [-0.1029 -0.1173  0.5641]
 [-0.0063 -0.3066  0.6868]
 [ 0.8055 -0.3446  0.346  ]]

-----
b^(1): Bias for Hidden Layer
-----
NumPy shape:      (5,)
Mathematical shape: 5 × 1 (column vector)
Values: [0. 0. 0. 0. 0.]

-----
W^(2): Weights from Hidden to Output Layer
-----
NumPy shape:      (1, 5)
Mathematical shape: 1 × 5

Values (rounded to 4 decimals):
[[-0.2241  0.0812  0.1286 -0.6377  0.032  ]]

-----
b^(2): Bias for Output Layer
-----
NumPy shape:      (1,)
Mathematical shape: 1 × 1 (scalar)
Values: [0.]
=====
```

Cell 4:

```
=====
FIRST DATA POINT (for all manual calculations)
=====

-----
Feature Vector x
-----
NumPy shape:      (3,)
Mathematical shape: 3 × 1 (column vector)

Values: [5.1 3.5 0.2]
  x[0] (sepal length): 5.1
  x[1] (sepal width):  3.5
  x[2] (petal width):  0.2

-----
Target Value y
-----
Mathematical shape: scalar
Value (petal length): 1.4
```

Cell 5:

```
=====
FORWARD PASS: Hidden Layer Pre-Activations
=====

Operation:  $a^{(1)} = W^{(1)} @ x + b^{(1)}$ 

-----
Dimensional Analysis
-----
 $W^{(1)}$ : 5 × 3 (mathematical), shape (5, 3) (NumPy)
x:      3 × 1 (mathematical), shape (3,) (NumPy)
 $b^{(1)}$ : 5 × 1 (mathematical), shape (5,) (NumPy)
-----
 $a^{(1)}$ : 5 × 1 (mathematical), shape (5,) (NumPy)

-----
Pre-Activation Values  $a^{(1)}$ 
-----
Values (rounded to 4 decimals):
[-0.2229 -2.0714 -0.8226 -0.9679  2.9711]

-----
Interpretation
-----
Each  $a_i^{(1)}$  is the weighted sum of inputs to hidden neuron i.
These values are BEFORE applying the activation function.
```

Cell 6:

```
=====
FORWARD PASS: Hidden Layer Activations
=====

Operation:  $z^{(1)} = \tanh(a^{(1)})$ 
(Applied element-wise to each component)

-----

Dimensional Analysis
-----
Input  $a^{(1)}$ : 5 × 1 (mathematical), shape (5,) (NumPy)
Output  $z^{(1)}$ : 5 × 1 (mathematical), shape (5,) (NumPy)

-----

Activation Values  $z^{(1)}$ 
-----
Values (rounded to 4 decimals):
[-0.2193 -0.9687 -0.6765 -0.7478  0.9948]

-----

Interpretation
-----
Each  $z_i^{(1)}$  is the activated output of hidden neuron  $i$ .
The tanh function squashes values to the range (-1, 1).
This introduces non-linearity, enabling complex pattern learning.
```

Cell 7:

```
=====
FORWARD PASS: Output Layer
=====

Operation:  $a^{(2)} = W^{(2)} @ z^{(1)} + b^{(2)}$ 
Output activation: Linear (identity), so  $y_{\text{hat}} = a^{(2)}$ 

-----

Dimensional Analysis
-----
 $W^{(2)}$ : 1 × 5 (mathematical), shape (1, 5) (NumPy)
 $z^{(1)}$ : 5 × 1 (mathematical), shape (5,) (NumPy)
 $b^{(2)}$ : 1 × 1 (scalar), shape (1,) (NumPy)

-----
 $a^{(2)}$ : 1 × 1 (scalar), shape (1,) (NumPy)
-----

Output Values
-----
Pre-activation  $a^{(2)}$ :    0.3922
Network prediction  $y_{\text{hat}}$ : 0.3922
True value  $y$ :          1.4000
Prediction error:        -1.0078

-----

Interpretation
-----
The network predicts petal length = 0.3922 cm
The true petal length is 1.4000 cm
→ The network underestimated by 1.0078 cm
```

Cell 8:

```
=====
FORWARD PASS: Loss Computation
=====

Mean Squared Error (MSE) Loss:
  E = (1/2) * (y_hat - y)^2

-----

Step-by-Step Calculation
-----

1. Prediction error: y_hat - y = 0.3922 - 1.4000 = -1.0078
2. Squared error:      (-1.0078)^2 = 1.0156
3. Loss:                E = (1/2) * 1.0156 = 0.5078

-----

Interpretation
-----

The loss E = 0.5078 measures prediction quality.
→ Lower loss means better predictions.
→ The factor of 1/2 simplifies the gradient to (y_hat - y).

=====
✅ CHECKPOINT 2 COMPLETE: Forward pass finished.
=====
```

Cell 9:

```
=====
BACKPROPAGATION: Output Layer Gradients
=====

Gradient of Loss w.r.t. Output
-----
Derivation: dE/da^(2) = d/da^(2) [(1/2)(a^(2) - y)^2]
              = (1/2) * 2 * (a^(2) - y) * 1
              = a^(2) - y
Result: dE/da^(2) = -1.0078

-----

Error Signal at Output
-----
delta^(2) = dE/dz^(2) = dE/da^(2) (output activation is linear)
Result: delta^(2) = -1.0078

-----

Gradient w.r.t. W^(2)
-----
Derivation: dE/dW^(2) = (y_hat - y) * (z^(1))^T

NumPy shape:      (1, 5)
Mathematical shape: 1 x 5
Should match W^(2): (1, 5) ✓

Values (rounded to 4 decimals):
[[ 0.221  0.9763 0.6818 0.7536 -1.0025]]

-----

Gradient w.r.t. b^(2)
-----
Derivation: dE/db^(2) = dE/da^(2) = y_hat - y
Result: dE/db^(2) = -1.0078

-----

Interpretation
-----
dE/da^(2) = -1.0078 tells us:
→ Output is too LOW; increase it to reduce loss
```


Cell 10:

```
=====
BACKPROPAGATION: Hidden Layer Error Signal
=====

-----
Error Propagation via Chain Rule
-----
We compute:  $\delta^{\wedge}(1) = dE/dz^{\wedge}(1)$ 
Using:       $\delta^{\wedge}(1) = (W^{\wedge}(2))^T * dE/da^{\wedge}(2)$ 

-----
Dimensional Analysis
-----
W^{\wedge}(2): 1 x 5 (mathematical), shape (1, 5) (NumPy)
(W^{\wedge}(2))^T: 5 x 1 (mathematical), shape (5, 1) (NumPy)
dE/da^{\wedge}(2): scalar = -1.0078

-----
delta^{\wedge}(1): 5 x 1 (mathematical), shape (5, 1) (NumPy)

-----
Why the Transpose?
-----


- $W^{\wedge}(2)$  is a row vector (1 x 5)
- We need a column vector (5 x 1) to match  $z^{\wedge}(1)$
- Transposing gives us the correct shape
- This is the Jacobian transpose for backpropagation



-----
delta^{\wedge}(1) = dE/dz^{\wedge}(1) Values
-----
Values (rounded to 4 decimals):
[ 0.2258 -0.0818 -0.1296  0.6427 -0.0323]

-----
Interpretation
-----
Each  $\delta_j^{\wedge}(1) = w_{\{1j\}}^{\wedge}(2) * (y_{\text{hat}} - y)$ 
This shows how much each hidden neuron contributed to output error.
Larger magnitude → stronger influence on the output.
```

Cell 11:

```
=====
BACKPROPAGATION: Hidden Layer Pre-Activation Gradients
=====

-----
Activation Function Derivative
-----
For tanh:  $h'(a) = 1 - \tanh^2(a) = 1 - z^2$ 
(Derived using quotient rule in Checkpoint 3)

h'(a^{\wedge}(1)) values (rounded to 4 decimals):
[0.9519 0.0615 0.5423 0.4488 0.0105]

-----
Gradient Through Activation
-----
Chain rule:  $dE/da^{\wedge}(1) = \delta^{\wedge}(1) \odot h'(a^{\wedge}(1))$ 
( $\odot$  denotes element-wise multiplication, Hadamard product)

-----
Dimensional Analysis
-----
delta^{\wedge}(1): 5 x 1 (column vector)
h'(a^{\wedge}(1)): 5 x 1 (column vector)

-----
Result: 5 x 1 (column vector)

-----
dE/da^{\wedge}(1) Values
-----
Values (rounded to 4 decimals):
[ 0.2149 -0.0005 -0.0703  0.2833 -0.0003]

-----
Interpretation
-----
dE/da_i^{\wedge}(1) tells us how changing pre-activation  $a_i^{\wedge}(1)$  affects loss.
The activation derivative  $h'(a)$  modulates gradient flow backward.
If  $h'(a)$  is small → gradient flow is diminished (vanishing gradient).
```

Cell 12:

```
-----
Gradient w.r.t. W^(1)
-----
Derivation: dE/dW^(1) = outer_product(dE/da^(1), x)
Mathematical: (5 x 1) ⊗ (1 x 3) = (5 x 3)

NumPy shape:      (5, 3)
Should match W^(1): (5, 3) ✓

Values (rounded to 4 decimals):
[[ 1.0962e+00  7.5230e-01  4.3000e-02]
 [-2.5700e-02 -1.7600e-02 -1.0000e-03]
 [-3.5850e-01 -2.4600e-01 -1.4100e-02]
 [ 1.4448e+00  9.9160e-01  5.6700e-02]
 [-1.7000e-03 -1.2000e-03 -1.0000e-04]]

-----
Gradient w.r.t. b^(1)
-----
Derivation: dE/db^(1) = dE/da^(1)
Shape: (5,)

Values (rounded to 4 decimals):
[ 0.2149 -0.005  -0.0703  0.2833 -0.0003]

-----
Complete Chain Rule Example: dE/dw_{12}^(1)
-----
Weight from input 2 (sepal width) to hidden neuron 1

Full chain:
dE/dw_{12}^(1) = dE/da^(2) * da^(2)/dz_1^(1) * dz_1^(1)/da_1^(1) * da_1^(1)/dw_{12}^(1)

Factor 1: dE/da^(2)           = -1.0078
Factor 2: da^(2)/dz_1^(1)     = w_{11}^(2) = -0.2241
Factor 3: dz_1^(1)/da_1^(1)   = h'(a_1^(1)) = 0.9519
Factor 4: da_1^(1)/dw_{12}^(1) = x_2 = 0.2000

Chain rule product:           = 0.0430
From gradient matrix dE/dW^(1)[0,1]: 0.0430
Match: True ✓

-----
Interpretation
-----
dE/dW^(1) tells us how to adjust each weight to reduce loss.
→ Larger magnitude = stronger influence on loss
→ Positive gradient = decrease weight to reduce loss
→ Negative gradient = increase weight to reduce loss
```

Cell 13:

```
-----
Pre-Activation Gradients dE/da^(1)
-----
Values:
  dE/da_1^(1) = 0.2149
  dE/da_2^(1) = -0.0050
  dE/da_3^(1) = -0.0703
  dE/da_4^(1) = 0.2833
  dE/da_5^(1) = -0.0003
-----

Gradients w.r.t. W^(1)
-----
Shape: (5, 3)

Values:
[[ 1.0962e+00  7.5230e-01  4.3000e-02]
 [-2.5700e-02 -1.7600e-02 -1.0000e-03]
 [-3.5850e-01 -2.4600e-01 -1.4100e-02]
 [ 1.4448e+00  9.9160e-01  5.6700e-02]
 [-1.7000e-03 -1.2000e-03 -1.0000e-04]]
-----

Gradients w.r.t. b^(1)
-----
Values:
  dE/db_1^(1) = 0.2149
  dE/db_2^(1) = -0.0050
  dE/db_3^(1) = -0.0703
  dE/db_4^(1) = 0.2833
  dE/db_5^(1) = -0.0003
=====
VERIFICATION CHECKLIST
=====
```